

Simulación de Entrevista Técnica — Kubernetes

Guía de estudio en formato entrevista · manifiestos, Deployments/StatefulSets, Services e Ingress para el sistema POS

Guía de estudio en formato entrevista sobre **Kubernetes** aplicado a este proyecto POS (manifiestos en `k8s/` : namespace, MongoDB, PostgreSQL, Kafka KRaft, microservicios y frontends). Cada sección incluye la **pregunta del entrevistador**, una **respuesta modelo** y **follow-ups**.

Formato sugerido: 50–60 min. Bloques: fundamentos (10'), workloads (15'), red/servicios (10'), config/estado (10'), operación y escalado (10').

0. Warm-up — K8s en el proyecto

P ¿Cómo se despliega el sistema en Kubernetes?

R Los manifiestos viven en `k8s/` y se aplican en orden con `deploy.sh / deploy.bat` : 00-namespace , 01-mongodb (StatefulSet), 02-postgres (StatefulSet), 03-kafka (KRaft, sin ZooKeeper), 04-microservices (los 6 backends) y 05-frontend . Las bases de datos y Kafka son **stateful** (StatefulSet + volúmenes), los servicios y frontends son **stateless** (Deployment). El namespace aísla los recursos del POS.

1. Fundamentos

P1 ¿Qué problema resuelve Kubernetes frente a Docker Compose?

R Compose orquesta contenedores en **una sola máquina** (ideal para desarrollo). Kubernetes orquesta contenedores en un **clúster de nodos** con **auto-recuperación** (reinicia/reprograma pods caídos), **escalado** (manual y automático), **rolling updates** sin downtime, **service discovery** y balanceo nativos, y gestión declarativa del estado deseado. Es el paso de "correr contenedores" a "operar un sistema distribuido en producción".

P2 ¿Qué es el modelo declarativo y el reconciliation loop?

R Declaras el **estado deseado** en manifiestos YAML (p. ej. "quiero 3 réplicas"). Los **controladores** ejecutan un **reconciliation loop**: comparan el estado actual con el deseado y actúan para converger (si hay 2 pods, crean 1 más; si sobra, lo eliminan). Por eso K8s es **auto-reparador**: si un pod muere, el controlador lo recrea sin intervención.

FOLLOW-UP ¿Componentes del control plane? — `kube-apiserver` (API), `etcd` (estado), `scheduler` (asigna pods a nodos), `controller-manager` (los loops). En cada nodo: `kubelet` y `kube-proxy`.

2. Workloads: Pods, Deployments, StatefulSets

P3 ¿Qué es un Pod y por qué no se despliegan pods directamente?

R Un **Pod** es la unidad mínima desplegable: uno o más contenedores que comparten red y volúmenes. No se crean pods "sueltos" porque son **efímeros** y no se recuperan solos; se usan controladores (**Deployment**) que los gestionan, mantienen el número de réplicas y hacen rollouts.

P4 ¿Deployment vs StatefulSet? ¿Cuándo cada uno aquí?

R **Deployment**: pods **intercambiables/stateless**, nombres aleatorios, rolling updates; ideal para los **servicios y frontends** del POS. **StatefulSet**: pods con **identidad estable** (nombre ordinal `-0`, `-1`), almacenamiento persistente por pod y arranque/terminación ordenados; necesario para **MongoDB, PostgreSQL y Kafka**, donde cada instancia tiene datos y una identidad de red propia.

FOLLOW-UP ¿Por qué Kafka en KRaft en K8s? — KRaft elimina ZooKeeper (Kafka gestiona su propio consenso de metadatos con Raft), simplificando el despliegue: menos componentes stateful que operar.

3. Red: Services e Ingress

P5 ¿Qué es un Service y qué tipos hay?

R Un **Service** da una **IP/DNS estable** y balanceo sobre un conjunto de pods (seleccionados por labels), desacoplando a los clientes de las IPs efímeras de los pods. Tipos: **ClusterIP** (interno, por defecto), **NodePort** (expone puerto en cada nodo), **LoadBalancer** (LB del cloud). El DNS interno (`venta-service.namespace.svc`) hace innecesario Eureka dentro del clúster.

P6 ¿Service vs Ingress?

R Un **Service** expone a nivel L4 (TCP/UDP) dentro (o hacia fuera con LoadBalancer). Un **Ingress** opera en L7 (HTTP/HTTPS): enruta por host/path, termina TLS y consolida el acceso externo por un solo punto, apoyado en un *ingress controller* (Nginx, Traefik). En este sistema, un Ingress podría cumplir parte del rol del API Gateway para el tráfico entrante.

4. Configuración y estado

P7 ¿ConfigMap vs Secret? ¿Dónde van las variables del `.env` ?

R **ConfigMap**: configuración no sensible (URLs de Eureka/Kafka, puertos). **Secret**: datos sensibles (`JWT_SECRET` , credenciales de Postgres/Mongo), codificados base64 y con control de acceso RBAC. Se inyectan como **variables de entorno** o volúmenes montados. Esto reemplaza el `.env` del Compose y evita hornear secretos en imágenes.

FOLLOW-UP ¿Base64 = cifrado? — No, base64 es solo codificación. Para seguridad real se habilita **encryption at rest** en etcd y/o un gestor externo (Sealed Secrets, Vault, cloud KMS).

P8 ¿Cómo se persisten los datos? PV, PVC y StorageClass.

R Un **PersistentVolumeClaim** (PVC) es la *solicitud* de almacenamiento que hace el pod; se satisface con un **PersistentVolume** (PV), aprovisionado dinámicamente vía **StorageClass** (disco del cloud). Los StatefulSets de Mongo/Postgres/Kafka usan `volumeClaimTemplates` para dar a **cada pod** su propio PVC, de modo que los datos sobreviven a reprogramaciones del pod.

5. Salud, escalado y operación

P9 ¿Diferencia entre liveness, readiness y startup probes?

R **Liveness**: ¿el pod está vivo? Si falla, K8s lo **reinicia**. **Readiness**: ¿está listo para recibir tráfico? Si falla, lo **saca del Service** (sin reiniciar). **Startup**: da margen a apps de arranque lento (¡Spring Boot!) antes de que liveness empiece a contar. Se apuntan a `/actuator/health`. Sin readiness bien configurado, el tráfico llegaría a un pod que aún arranca.

P10 ¿Cómo escalas y por qué importan requests/limits?

R Escalado horizontal manual (`kubectl scale`) o con **HPA** (Horizontal Pod Autoscaler) según CPU/memoria o métricas. Los **resource requests** guían al scheduler (cuánto reservar) y los **limits** ponen el techo. Es el equivalente K8s a los límites de CPU del Compose: dado que Spring Boot es CPU-bound al arrancar, hay que darle requests suficientes o el arranque se ralentiza.

FOLLOW-UP ¿Qué pasa si un pod excede su limit de memoria? — Es **OOMKilled** (terminado por el kernel) y reiniciado. Si excede CPU, se le **throttlea** (no lo matan).

P11 ¿Cómo hace K8s un despliegue sin downtime?

R Con **rolling update**: crea pods nuevos y retira los viejos gradualmente respetando `maxSurge` / `maxUnavailable`, y solo enruta tráfico a pods `ready`. Si algo falla, `kubectl rollout undo` revierte. Alternativas: **blue-green** o **canary** para exponer la versión nueva a un porcentaje del tráfico primero.

6. Diseño abierto / escenarios

P12 Migras de Compose a K8s. ¿Qué cambia respecto a Eureka y el gateway?

R El **service discovery** lo puede dar el DNS del clúster (Service), haciendo Eureka opcional. El acceso externo puede pasar por un **Ingress** en vez del gateway, o mantener el gateway como un Deployment más. La config del `.env` pasa a **ConfigMaps/Secrets**, y los límites de CPU/memoria a **requests/limits**. Bases y Kafka se vuelven **StatefulSets con PVC**.

P13 Un pod entra en `CrashLoopBackOff` . ¿Cómo lo diagnosticas?

R `kubectl describe pod` (eventos, motivo de fallo, `OOMKilled`), `kubectl logs --previous` (logs del contenedor que murió), revisar **probes** (una liveness demasiado agresiva reinicia una app que aún arranca), **requests/limits** (falta de memoria) y dependencias (¿la BD/Kafka están listas?). El `BackOff` es el retardo exponencial entre reintentos de reinicio.

Apéndice — Chuleta rápida

Tema	Punto clave
Vs Compose	Orquesta un clúster con auto-recuperación, escalado y rolling updates
Declarativo	Estado deseado + reconciliation loop → auto-reparación
Pod	Unidad mínima; efímero; gestionado por controladores (Deployment)
Deployment vs StatefulSet	Stateless intercambiable vs identidad+almacenamiento estables (Mongo/PG/Kafka)
Service	IP/DNS estable + balanceo; ClusterIP/NodePort/LoadBalancer; sustituye a Eureka
Ingress	L7: enruta por host/path, termina TLS; puede hacer de gateway de entrada
Config	ConfigMap (no sensible) + Secret (sensible; base64 ≠cifrado)
Estado	PVC → PV vía StorageClass; volumeClaimTemplates por pod en StatefulSet
Probes	liveness (reinicia) / readiness (saca del Service) / startup (arranque lento)
Escalado	HPA + requests/limits; exceso de memoria → OOMKilled
Deploy	Rolling update (maxSurge/maxUnavailable); rollout undo; canary/blue-green